

RECURSION

Key ideas: base case (immediate, obvious); recursive case calls self on smaller problem; each call has own stack frame; must approach base case. **Largest:** compare head vs. largest(rest). **Reverse:** remove head, recurse, append head at end. **Member-of:** head==x OR member(rest). **Multiply** i×j: base j=0; else i+mult(i,j-1).

TOWERS OF HANOI

Move n disks src→dst using tmp:
1. solve(n-1, src, dst, tmp) — move top n-1 via dst
2. move(src→dst) — the base disk
3. solve(n-1, tmp, src, dst) — move from tmp
Total moves: 2ⁿ-1. Pins swap roles (src/tmp/dst) each call. "Very difficult without recursion!"

FIBONACCI DISASTER

f(n)=f(n-1)+f(n-2): exponential redundant calls; bogs down at large n. Use **iteration** instead. Recursion is powerful but use appropriately.

QUICKSORT

(1961, "most elegant algorithm") **Partition:** pick middle as pivot (A); park at right by swap (B); set i=left-1, j=right; loop: move i right while A[i]<pivot; move j left while A[j]>pivot; if i<j swap A[i],A[j]; swap pivot back at i. **Sort:** base: size<2 return; size=2 swap if needed; else partition+recurse left & right.

DYNAMIC BACKTRACKING

Decision tree: try a path; backtrack on dead end; try next path. Base = no further paths (dead end) or solution found. Continue after solution to find all solutions. **N-Queens (8×8):** solve column-by-column L→R. is_safe(row,col) checks left-side attacks. find_solutions() recurses right. **92 solutions. Sudoku:** brute-force 1-9 per cell; L→R top→bottom; recursively check ahead; backtrack after trying all 9; backtrack to previous cell.

MULTITHREADING

Thread: path through code; own stack+registers; shares code/data/resources. **Concurrency** (interleaved, 1 CPU) vs. **Parallelism** (simultaneous, multi-CPU). **Critical region:** code accessing shared resource. **Mutual exclusion:** only 1 thread in critical region at a time. **mutex:** lock() blocks if locked; unlock() on exit. **lock_guard<mutex>:** RAIL — auto-lock on construct, auto-unlock on scope exit; prevents forgotten-unlock deadlock. **yield:** this_thread::yield() voluntarily relinquish CPU. sleep_for(chrono::seconds(n)). **Reader-Writer:** shared_mutex. Writers: unique_lock<shared_mutex> (exclusive — no readers or writers). Readers: shared_lock<shared_mutex> (concurrent reads OK; blocks writers). **atomic<T>:** thread-safe without mutex; store(), load(), -- all atomic. **condition_variable:** cv.wait(lock) atomically unlocks+sleeps; re-locks when notified. Always use while() loop (spurious wakeup). cv.notify_all() wakes all. cv.notify_one() wakes one.

PRODUCER-CONSUMER

Queue with capacity CAP. Producers wait on producer_cv when full. Consumers wait on consumer_cv when empty. Producer notifies consumers when size becomes 1 (just not-empty). Consumer notifies producers when size becomes CAP-1 (just not-full). When all producers done, notify consumers.

DINING PHILOSOPHERS

5 philosophers, 5 chopsticks (one between each pair). Need both adjacent; pick up both at once (cannot hold one and wait). Loop: think→wait→fill bowl→eat. **Deadlock:** all hold one, wait for other. **Fix:** consistent lock ordering (always acquire lower-numbered chopstick first).

auto & decltype

auto: type inference from initializer. **decltype(x):** yields static type of expression x; declare same-type var without knowing the type explicitly.

WELL-DESIGNED SOFTWARE

Meets requirements · Reliable (no surprises, no hidden perf issues) · Flexible+scalable (add features without complexity) · Maintainable · Uses good design+patterns · Saves time/cost. Design-code-test iterations ("Mozart" analogy — iterative, not big-bang).

OOP DESIGN

Encapsulation: data+behavior in class; public/private; isolates changes. **Abstraction:** ignore irrelevant details. **Inheritance:** subclass adds/overrides state+behavior. **Polymorphism:** base ptr calls correct subclass method at runtime via virtual. **dynamic_cast<T*>(ptr):** type-check+convert; returns nullptr if wrong type. **Abstract class:** ≥1 pure virtual (=0); cannot instantiate; subclasses must implement. **Interface class:** all pure virtual, no member data. **virtual destructor** required when subclasses exist. **Override** (same signature, different class) vs. **Overload** (same name, different params, same scope). **Liskov (LSP):** subclass must be substitutable for superclass without breaking behavior. **Violation:** CircularBuffer::vector exposes invalid methods (at(), erase()). **Fix:** has-a not is-a (member vector<int> buffer). **Code to Interface:** declare as base ptr, assign subclass at runtime.

DESIGN SMELLS

Leaking changes: change in Car forces change in Driver (key→button). **God class:** AutomobileApp has too many responsibilities (drive+maintain+wash). **Hardcoded concrete:** Cat *pet = new Cat(). **Least surprise violation:** Date MONTH_NAMES[12] off-by-one (month=9→OCT not SEP). **DRY violation:** same code repeated across multiple classes.

DESIGN PRINCIPLES (9)

Principle	Key Idea
SRP	One primary responsibility; cohesive; easy use+maintain
Encapsulate What Varies	Isolate changeable code; changes don't leak to other classes
Delegation	Move work to the most suitable class; requester has no HOW dependency
Law of Demeter	Call only: own member vars, params, locally created; NOT returned obj (a.b() .c() =small); return copies not ptrs
OCP	Closed to modification; open for extension via subclasses
Code to Interface	Use base/interface ptr not specific subclass ptr
DRY	No duplicate code; share as function/class
Defensive Programming	Runtime param checks; create only valid objects; assert()
Programming by Contract	Precond=caller's responsibility; Postcond=fn's responsibility; Invariant=true after ctor + every mutation

Also: **Favor Composition over Inheritance** (has-a often better). **Lazy Evaluation. Refactoring** (improve design without changing functionality).

REQUIREMENTS

Functional: what system *shall* do / allow users to do. **Nonfunctional:** usability, reliability, performance, supportability — equally important. Use **"shall"/"must"** (mandatory); **"should"** = wish list. Sources: interview, observe, stated+implied. **Iterative** — show prototypes; customers don't always know what they want. **I18N** (Internationalization, 18 letters) — design for multiple locales. **L10N** (Localization, 10 letters) — adapt language, dates, monetary, encoding. **V&V: Validate** = building the right app (customer confirms); **Verify** = building the app right (developers test).

Quality	Meaning
Clear	Non-technical, jargon-free language
Consistent	No contradictions between requirements
Correct	Each requirement must be accurate
Complete	No gaps; all scenarios covered
Realistic	Satisfiable with available resources
Verifiable	Testable — can confirm it's met
Traceable	Each req ↔ design feature

UML DIAGRAMS

Class: name; vars (+pub/-priv/#prot); fns. **6 relationship lines:** dependency (dashed arrow), transient dependency, aggregation (◊— has vector-of; part can exist independently), composition (◆— part cannot exist outside container), generalization (→superclass), realization (· →implements interface). Multiplicity on arrows (1, *, 1..n). **Use Case:** oval=use case; stick figure=actor (person or external system); rectangle=system boundary. **State:** rounded boxes=states; arrows=transitions with events; [guards] in brackets; ●=start state; ○=end state. **Sequence:** lifelines (dashed vertical lines); activation boxes (rectangles); labeled horizontal arrows=method calls; time flows top→bottom.

USE CASE DESCRIPTION (10 fields)

Name (verb-noun); **Goal; Summary** (1-2 sent.); **Actors; Preconditions** (must be true before); **Trigger** (actor action that starts it); **Primary sequence** (≤10 numbered steps); **Alternate sequences** (exception paths, numbered); **Postconditions; Nonfunctional requirements.** Focus on WHAT not HOW; "black box" view; no GUI or implementation details.

FUNCTIONAL SPEC & DESIGN SPEC

Functional Spec (External Ref Spec): product name, problem statement, objectives, functional reqs, nonfunctional reqs, use cases. No impl details. Customer reads/approves → **validates** (building right app). **Design Spec** (Internal Maintenance Spec): technical+impl details, UML class/state/sequence diagrams. Developers → **verify** (building app right).

REQUIREMENTS ANALYSIS (textual)

Nouns → **Classes** (member vars for state). **Verbs** → **Methods** (transitive verbs acting on objects → member fns; assign cohesively per SRP). **Noun-Class Table:** noun, class, states/vars. **Verb-Behavior Table:** verb, class, member fn name.

LAMBDA EXPRESSIONS

Syntax: [capture] (params) -> ret {body}. No capture: {} (const T& x) {...}. By ref: [&var]; by value: [var]; all by ref: [&]. **Predicate:** bool fn whose return depends on params. std::function<bool>(const T&) > for lambdas with capture. Use with STL: sort, find_if, count_if. Example: auto f = [] (int x) {return x*2;;};

FUNCTORS (Function Objects).

Class overloading operator()(). Stores state across calls in member vars (unlike regular functions). Can be stored, passed, returned. RandomInt ri(5,12); ri() -> next random int in [5,12]. Summation adder; adder(3); -> accumulates. Syntax: first () =operator name, second () =parameter list.

MOVE SEMANTICS & RULE OF BIG FIVE

lvalue: named variable (can be left of =). **rvalue:** temporary expression, about to be destroyed. Moving rvalue = "steal" resources O(1) vs. copy O(n). std::move(x): cast lvalue to rvalue reference (enables move). **Rule of Big Five:** if you define any one -> define all five:

1. **Destructor** ~T() — free resources
2. **Copy ctor** T(const T&) — deep copy
3. **Copy assign** T& operator=(const T&)
4. **Move ctor** T(T&&) — steal + set source to nullptr
5. **Move assign** T& operator=(T&&)

Move ctor/assign steal resources and set source to valid empty state (nullptr). Avoids costly copies of temporaries (e.g., push_back(first+" "+last)).

SMART POINTERS

Raw ptrs: memory leak, double delete, no ownership clarity. **unique_ptr<T>:** exclusive ownership; auto-deleted on scope exit or reset; cannot copy (only move via std::move); source set to nullptr. **shared_ptr<T>:** shared ownership via ref count; auto-deleted when count reaches 0; copy increments count; reset() decrements; deleted exactly once.

VECTORS & PERFORMANCE

Growing a vector copies all elements O(n) each time. Use reserve(n) to pre-allocate capacity. Store T* not T to avoid copy overhead on insert/remove (only pointer copies needed). Remove shifts all following elements O(n).

MVC ARCHITECTURE

Model: persistent data (memory/files/DB). **View:** UI components (fields, buttons, menus). **Controller:** coordinates Model+View; dispatches user input to Model. Qt6 framework; inversion of control; loose coupling among all three. User cannot directly modify Model.

DESIGN PATTERNS — CLASSIFICATION

Creational (3)	Structural (4)	Behavioral (6)
Factory Method	Adapter	Template Method
Abstract Factory	Facade	Strategy
Singleton	Composite	Iterator
	Decorator	Visitor
		Observer
		State

TEMPLATE METHOD DP

Intent: fixed algorithm skeleton in base class (the "template method"); varying steps are abstract methods in subclasses. **Structure:** GameReport.generate_report() calls steps in order; base implements common steps (print_header/footer); abstract steps (acquire_data, analyze_data, print_report) in BaseballReport, VolleyballReport. **Mechanism:** Inheritance. **Keywords:** "algorithm outline", "fixed sequence", "outline of steps". **vs. Strategy:** TM=inheritance-fixed outline with varying steps; Strategy=composition+entire algorithm swapped.

STRATEGY DP

Intent: family of interchangeable algorithms; interface per family; concrete classes implement; context has-a strategy ptr set at runtime. **Structure:** Sport has player_strategy + venue_strategy ptrs; interfaces PlayerStrategy and VenueStrategy; concretes: BaseballPlayers, Stadium, OpenField. **Mechanism:** Composition. **Keywords:** "interchangeable", "runtime selection", "family of algorithms". **vs. TM:** Strategy swaps entire algorithm; TM varies only steps within fixed outline.

FACTORY METHOD DP

Intent: creator superclass has abstract factory method; subclass decides which product to instantiate. **Structure:** Creator=AthleticsDept; ConcreteCreators=VarsityDept, IntramuralDept; factory method=create_sport(); Product=Sport. Superclass delegates object creation to subclasses. **Mechanism:** Inheritance. **Keywords:** "subclass decides", "single product type", "factory method". **vs. Abstract Factory:** FM=single product type; AF=families of related products; AF prevents mixing families.

ABSTRACT FACTORY DP

Intent: interface creates families of related objects; concrete factory creates one consistent family; prevents mixing families. **Structure:** AbstractFactory=ProvisionsFactory; Concretes=VarsityFactory, IntramuralFactory; each creates a family: make_players(), make_venue(). **Mechanism:** Composition. **Keywords:** "product family", "consistency", "prevents mixing". **vs. FM:** AF ensures family consistency; FM just delegates single-type creation to subclass.

ADAPTER DP

Intent: make incompatible interfaces work together; wrap legacy/external code. **Structure:** Target interface=AttendanceData; Adapter=FootballData implements Target + aggregates Adaptee=FootballInfo; converts calls. **Class Adapter variant:** uses multiple inheritance instead of aggregation; can override adaptee methods. **Keywords:** "incompatible interface", "legacy code", "wrapper". **vs. Facade:** Adapter=make incompatible work together; Facade=simplify complex subsystem.

FACADE DP

Intent: simple high-level interface to complex subsystem; hide internal complexity. **Structure:** Facade=FundRaiser delegates to subsystem classes (Alumni, BoosterClubs, Students); clients (AthleticsDept, Administration) only call Facade. **Keywords:** "simplify", "hide complexity", "front door", "single interface to subsystem".

SINGLETON DP

Intent: exactly one instance of a class; global access point. **Structure:** private constructor; private static ptr single_instance; public static get_singleton() — lazy creation (creates if null, returns existing if not); deleted copy ctor + copy assign. **Thread safety issue:** multiple threads may each create instance if single_instance==nullptr simultaneously -> fix with mutex or static local variable. **Keywords:** "one instance", "global access", "private constructor", "lazy construction".

COMPOSITE DP

Intent: uniform treatment of individual objects and compositions; part-whole hierarchy. **Structure:** Component interface=ProvisionItem; Leaf=Ball, Bat (no children); Composite=ProvisionGroup (is-a ProvisionItem AND has-a vector<ProvisionItem*>); get_cost() on composite recursively sums children. Disable add/remove on leaves. **Keywords:** "part-whole hierarchy", "tree structure", "recursive", "uniform treatment". **vs. Decorator:** Composite=multiple children+aggregate; Decorator=single child+add behavior.

DECORATOR DP

Intent: add responsibilities to an object dynamically; flexible alternative to subclassing. **Structure:** Component=Ticket; ConcreteComponent=BaseTicket; Decorator abstract (is-a Ticket, has-a Ticket* next); ConcreteDecorators=Party, VIP, Coupon; chain: get_cost() = next->get_cost() + my_cost. **Keywords:** "add responsibilities dynamically", "wrap", "chain of wrappers". **vs. Composite:** Decorator=single child+add behavior; Composite=multiple children+aggregate cost.

ITERATOR DP

Intent: traverse a collection without exposing its internal representation; decouple algorithm from collection type. **Structure:** Iterator interface: has_next(), next(); ConcreteIterators:ArrayIterator, VectorIterator, MapIterator (each with private cursor); Collection (Team subclasses) creates appropriate iterator; Client (TeamReport) uses only iterator interface. **Note:** C++ built-in iterators ≠ this pattern (not via common interface). **Keywords:** "traverse", "decouple collection", "has_next/next". **vs. Visitor:** Iterator=single traversal; Visitor=multiple different algorithms on same structure.

VISITOR DP

Intent: add new operations to a class hierarchy without modifying it; different algorithms on same object structure. **Structure:** Node interface has accept(Visitor&); accept impl: v.visit_X(this) then propagate to children; Visitor interface has visit_NodeType() per node type; ConcreteVisitors (ScoresReportVisitor, etc.) encapsulate algorithms. **Double dispatch:** node type + visitor type together determine behavior. **Analogy:** craftsmen (painter, plumber, electrician) visiting rooms — each craftsman does different work per room type. **Keywords:** "new operations without modifying", "node hierarchy", "double dispatch". **vs. Iterator:** Visitor=multiple algorithms; Iterator=single traversal method.

OBSERVER DP

Intent: publish-subscribe; 1-to-many dependency; loose coupling between subject and observers. **Structure:** Subject has vector<Observer*> observers; methods: subscribe()/unsubscribe()/notify() — notify calls update() on each; Subject stores current_event; Observer calls get_current_event() to pull data (pull model). BaseballReporter=Subject; LogReport, GraphReport, TableReport=Observers. **Keywords:** "publish-subscribe", "notify", "1-to-many", "event-driven", "loosely coupled".

STATE DP

Intent: state-dependent behavior; object appears to change its class; transitions are encapsulated. **Structure:** Context=TicketMachine has State* state; delegates all actions to current state. State interface has action methods. ConcreteStates (READY, VALIDATING, TICKET_SOLD, SOLD_OUT) encapsulate actions+transitions; each action fn returns next State*. StatesBlock aggregates all states (loose coupling). **UML state diagram** documents transitions. **Keywords:** "state-dependent behavior", "state machine", "encapsulated transitions", "alter behavior when state changes". **vs. Strategy:** State transitions internally based on actions; Strategy is selected externally.

PATTERN QUICK REFERENCE

Pattern	Cat	Mechanism	Confused with
Template Method	Beh	Inheritance	Strategy
Strategy	Beh	Composition	Template Method
Factory Method	Cre	Inheritance	Abstract Factory
Abstract Factory	Cre	Composition	Factory Method
Singleton	Cre	Static field	—
Adapter	Str	Aggregation/MI	Facade
Facade	Str	Delegation	Adapter
Composite	Str	Recursion/tree	Decorator
Decorator	Str	Chain/wrap	Composite
Iterator	Beh	Cursor	Visitor
Visitor	Beh	Double dispatch	Iterator
Observer	Beh	Notify/pull	—
State	Beh	Delegation	Strategy